Based on the commands you executed, here are the steps separated into **Developer A** and **Developer B**.

Developer A Commands

PowerShell

```
cd D:\newone\development\developerA

git clone https://github.com/nidhidhameliya/development.git

cd development

git branch developera

git checkout developera

# Edit README.md

git status

git add .

git commit -m "developer b changes commit"

git checkout main

git merge developera

git push origin main
```

Since Developer B had already pushed, Developer A got a push rejection:

PowerShell

```
git pull

# Resolve merge conflict in README.md

git add .

git commit -m "developer a story"

git push origin main
```

Developer B Commands

cmd

```
cd /d D:\newone\development\developerB

git clone https://github.com/nidhidhameliya/development.git

cd development

git branch developerB

git checkout developerB

# Edit README.md

git status

git add .

git commit -m "story of developer a"

git checkout main

git merge developerB

git push origin main
```

What happened?

1. **Developer B**
   - Created a branch.
   - Made changes.
   - Committed them.
   - Merged into `main`.
   - Successfully pushed to GitHub.
2. **Developer A**
   - Created another branch.
   - Made different changes.
   - Merged into local `main`.
   - Tried to push but GitHub rejected it because **Developer B had already updated** `main`.
   - Ran `git pull`.
   - Resolved the merge conflict in `README.md`.

- Committed the merge.
- Successfully pushed to GitHub.

This is the standard Git collaboration workflow demonstrating a **merge conflict** and its resolution.